Task for the Master's Thesis

Danje Petersen

November 21, 2025

1 Motivation

Almost every computer that is in use today provides multiple simultaneously running processors. To reap the benefits that multiple processors provide, developers design their programs to use concurrently running threads. Since multiple threads may operate on the same memory it is possible that on some memory location one thread writes a value that another thread needs, but never sees. In Figure 1 two threads each write a value on a memory location that the other one reads. Multiple executions are possible: If thread T_1 is executed completely before T_2 the result is different to when T_2 is executed completely before T_1. This is called a *data race*. Data races occur when two or more threads access the same memory space and at least one of them uses a write operation.

```
Init:x=y=0;
  T1   ||   T2
x=1;   ||  y=1;
a=y;   ||  b=x;
```

Figure 1: Example of a data race, where two threads use the same variables. The resulting values of a and b can be any combination of 1 and 0 depending on how the threads are executed.

CPU architectures and compilers may reorder the commands in such programs for performance reasons and interleave the two threads as they see fit. Since this permits different results for the same program and therefore unpredictable behavior, programming languages and cpu architectures provide memory consistency models (MCM). If the developer confines themselves to the rules of a programming language (e.g. Data-Race-Freedom (DRF)), they get certain guarantees in return for how the program will be compiled. These guarantees are in the form of a MCM. The memory consistency model of a CPU architecture describes how the hardware will behave if it runs a program. These memory consistency models describe how threads are allowed to interact with each other when it comes to shared memory. As can be seen in the example, the absence of a memory consistency model can change the

outcome of a program drastically. But the presence a memory consistency model will not automatically get rid of all problems. MCMs are defined by humans and can have bugs. It is furthermore possible that a MCM does not exactly reflect the behavior of the architecture. TODO ►*true?*◄ Consequently there is research on how MCMs can be verified and how the verification can be done effectively. In Section 2 we will look at current lines of research targeting memory consistency models and especially at the verification. In Section 3 I will describe what this master's thesis goals are and how this fits into the research of the last years. TODO ►*!?*◄

2 Current Research

There are currently several established lines of research in the field of weak memory. One obvious issue is the *consistency checking* of programs under a given memory model [13, 8]. Several automated tools that were developed in the last decade already handle this issue well. Examples are GenMC [8], DARTAGNAN [11] and Herd [1]. Some research groups instead examine the models themselves and their attributes [10] or the already mentioned operational models [5, 3]. Additionally, researchers are looking at ways of comparing [14, 12] and verifying [7, 2, 6] memory consistency models which are two closely related problems.

In 2014 Alglave et al. proposed an axiomatic framework for modelling weak memory and the corresponding language CAT [1] which has since then become the standard for formulating memory consistency models. Here, weakness is defined as accepting a larger number of possible executions of a program than stronger models. Axiomatic refers to the kind of model that can be formulated. Besides axiomatic ones, there are operational models which are abstractions of actual machines with idealised components [1]. Axiomatic models on the other side work by constraining relations on so-called events. Those events are the memory accesses in a program like reads and writes.

Some very basic examples for these relations are the *program order* (po) which represents the sequence of events of a single thread and *read-from* (rf), displaying for every read which write determined the value it sees. Axiomatic memory consistency models are defined on these and some more base relations, in the CAT language as can be seen in Figure 2.

While the tool that Alglave et al. developed allows to test programs under a given memory model, verifying these models is a different issue. Verification of the memory consistency model can be divided into five tasks. The first is to check the model against generic properties like having no out-of-thin-air values. The second task is to check properties that refer to operation's semantics like barrier monotonicity. Furthermore, it is one task to compare memory consistency models and another to create litmus tests that show whether a

$$
\begin{aligned}
\langle cat \rangle &:= \langle constraint \rangle \mid \langle cat \rangle \ \langle cat \rangle \\
\langle constraint \rangle &:= \texttt{acyclic} \ \langle rel \rangle \mid \texttt{irreflexive} \ \langle rel \rangle \mid \texttt{empty} \ \langle rel \rangle \\
\langle rel \rangle &:= \langle base_rel \rangle \mid \langle rel \rangle.\langle rel \rangle \mid \langle rel \rangle^* \mid \langle rel \rangle \cup \langle rel \rangle \mid \langle rel \rangle \cap \langle rel \rangle \\
&\quad \mid \langle rel \rangle^{-1} \mid \langle rel \rangle^+ \mid \langle rel \rangle? \mid \langle base_set \rangle \times \langle base_set \rangle \mid \langle rel \rangle \setminus \langle rel \rangle \mid \sim \langle rel \rangle \\
\langle base_rel \rangle &:= \text{po} \mid \text{rf} \mid \text{rmw} \mid \text{loc} \mid \text{ext} \mid \text{id} \mid 0 \\
\langle base_set \rangle &:= \text{R} \mid \text{W} \mid \text{F} \mid \text{B} \mid \text{M} \mid \text{IW} \mid \text{E}
\end{aligned}
$$

Figure 2: The CAT language when including the purple coloured terms, when excluding them it is the fragment called rational CAT [4]. $\langle rel \rangle^+$ is the transitive closure, $\langle rel \rangle?$ the reflexive closure, $\langle rel \rangle^*$ the reflexive and transitive closure, $\sim \langle rel \rangle$ the complement and 0 denotes the empty relation [1].

model is implemented by a language or architecture. The last task is to verify compiler transformations.

For the first task, the memory model in question can be compared to a model that is defined by just that property. The first tool to tackle this and the third task was KATER [7] which covered only a fragment of CAT that could be transformed into Kleene algebra with tests (KAT). This fragment excluded set difference, complement, intersections, converses and the construction of relations from sets.
The next year Koutsouridis et al. presented KATI [9] which is able to handle converses and some intersections but has not been implemented.

The first tool that addressed all intersections as well as inverses and construction of relations from sets (called rational CAT, see Figure 2) was RAT (Relational Algebra with Tableaus) which was developed by the supervisors of this master's thesis in 2025 [4]. Their tool is able to handle the first four tasks of the verification with only the compiler transformations being left as an open issue. In their yet to be published work, they present a logical theory of relations where the unsatisfiability of a formula is equivalent to the model fulfilling a set of assertions. By transforming the question of $M \subseteq M_{prop}$ into $\overline{M} \cap M_{prop} = \emptyset$ and then a formula $\neg\psi_M \wedge \psi_{M_{prop}}$ it proves whether the given model is stronger than the condition that is to be tested. The formula is then decomposed until there exists a satisfying model for the formula or it is deemed unsatisfiable.

Today there are many open questions in this field of research. Is it possible to automatically verify the whole of the CAT language or at least more than the rational CAT fragment? Can the popular models written in CAT be transformed into ones that are written in rational CAT? Is it possible to verify compiler transformations? This master's thesis will be about the issue of whether the proof system in [4] can be improved while sustaining the soundness and completeness.

3 Task

The decomposition used in [4] might result in infinite paths when unrolling Kleene stars. This leads to the authors using a cyclic proof system that returns to a node in the decomposition tree when it is equal to the current one. Even though the proofs might not be infinite, they can still get very large. The task for this master's thesis is to replace this cyclic proof system with one that uses invariants and is neither cyclic nor infinite. This is supposed to lead to smaller proofs. Additionally the new proof system is supposed to directly handle inclusion between memory models and should not need to resolve them to the event level. The goal of the thesis is to create a proof system that works reasonably well and ideally proof it's completeness and soundness.

References

[1] Jade Alglave, Luc Maranget, and Michael Tautschnig. "Herding Cats: Modelling, simulation, testing and data mining for weak memory". In: *ACM Transactions on Programming Languages and Systems* 36.2 (2014), pp. 1–74. ISSN: 0164-0925. DOI: 10.1145/2627752.

[2] P. S. Andrianov et al. "Software Model Checking for Memory Consistency Verification". In: *Programming and Computer Software* 51.5 (2025), pp. 305–313. ISSN: 1608-3261. DOI: 10.1134/S0361768825700185. URL: https://link.springer.com/article/10.1134/s0361768825700185.

[3] Simon Doherty et al. "Unifying Operational Weak Memory Verification: An Axiomatic Approach". In: *ACM Transactions on Computational Logic* 23.4 (2022), pp. 1–39. ISSN: 1529-3785. DOI: 10.1145/3545117.

[4] Jan Grünke and Roland Meyer. "RAT-CAT-SAT: Model Checking Memory Consistency Models". In: *unpublished* (2025).

[5] Dan Iorga, John Wickerson, and Alastair F. Donaldson. "Simulating Operational Memory Models Using Off-the-Shelf Program Analysis Tools". In: *IEEE Transactions on Software Engineering* 49.12 (2023), pp. 5084–5102. ISSN: 0098-5589. DOI: 10.1109/TSE.2023.3326056.

[6] Jake Kirkham et al. "Foundations of empirical memory consistency testing". In: *Proceedings of the ACM on Programming Languages* 4.OOPSLA (2020), pp. 1–29. DOI: 10.1145/3428294.

[7] Michalis Kokologiannakis, Ori Lahav, and Viktor Vafeiadis. "Kater: Automating Weak Memory Model Metatheory and Consistency Checking". In: *Proceedings of the ACM on Programming Languages* 7.POPL (2023), pp. 544–572. DOI: 10.1145/3571212.

[8] Michalis Kokologiannakis and Viktor Vafeiadis. "GenMC: A Model Checker for Weak Memory Models". In: *Computer Aided Verification*. Ed. by Alexandra Silva and K. Rustan M. Leino. Theoretical Computer Science and General Issues. Cham: Springer International Publishing and Imprint Springer, 2021, pp. 427–440. ISBN: 978-3-030-81685-8. DOI: 10.1007/978-3-030-81685-8{\textunderscore}20. URL: https://link.springer.com/chapter/10.1007/978-3-030-81685-8_20.

[9] Aristotelis Koutsouridis, Michalis Kokologiannakis, and Viktor Vafeiadis. "Automating Memory Model Metatheory with Intersections: Schloss Dagstuhl – Leibniz-Zentrum für Informatik". In: *LIPIcs, Volume 311, CONCUR 2024* 311 (2024). DOI: 10.4230/LIPICS.CONCUR.2024.33.

[10] Ori Lahav and Udi Boker. "What's Decidable About Causally Consistent Shared Memory?" In: *ACM Transactions on Programming Languages and Systems* 44.2 (2022), pp. 1–55. ISSN: 0164-0925. DOI: 10.1145/3505273.

[11] Hernan Ponce-de-Leon et al. "BMC with Memory Models as Modules". In: *Proceedings of the 18th Conference on Formal Methods in Computer-Aided Design (FMCAD 2018)*. Ed. by Nikolaj Bjørner and Arie Gurfinkel. Piscataway, NJ: IEEE, 2018, pp. 1–9. ISBN: 978-0-9835678-8-2. DOI: 10.23919/FMCAD.2018.8603021.

[12] Saksham Sharma, Vishwanath HV, and M. Uttara Kumari. "Meow: Memory Evaluation through Optimized Workloads - Synthesizing Litmus Tests for Memory Consistency Verification". In: *2023 IEEE 20th India Council International Conference (INDICON)*. IEEE, 2023, pp. 73–78. DOI: 10.1109/indicon59947.2023.10440892.

[13] Hünkar Can Tunç et al. "Optimal Reads-From Consistency Checking for C11-Style Memory Models". In: *Proceedings of the ACM on Programming Languages* 7.PLDI (2023), pp. 761–785. DOI: 10.1145/3591251.

[14] John Wickerson et al. "Automatically comparing memory consistency models". In: *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages*. Ed. by Giuseppe Castagna. ACM Digital Library. New York, NY: ACM, 2017, pp. 190–204. ISBN: 9781450346603. DOI: 10.1145/3009837.3009838.